Architectural Enhancement:

Gemini CLI

Group 9
The Unemployed

Video Link

Team Members

Kesiya Jacob
Group Leader - Abstract, Introduction, Risk of Enhancement, Plans for Testing

Gabrielle Garey
Presenter - Enhancement Proposal, Approach A, Approach B, Positives/ Negatives

Sophia Bushareb
SAAM Analysis, Effects on Enhancement, Use case 1

Grisha Tyukin
AI Collaboration, Interaction with other features, Data Dictionary, Naming Conventions

Ryan Walsh
Presenter - Impacted directories/files, use case 2, conclusion, AI report

Calvin Wong
Did not contribute

Our Proposal: Model Data Capture/ Replay

A feature that logs and replays Gemini CLI sessions, enabling debugging, auditing, and model comparison.

Our initial vision: User will type a flag into the terminal to signal they want to start a session. Gemini CLI will capture subsequent data about the session and user can see that data and choose to replay a session using a different LLM model for comparison.

Our motivation: Developers need to evaluate the effectiveness of the tools they use. This feature would fit in perfectly to the existing architecture to allow for more customization explainability for model performance.

Value + Benefits: Allows Gemini CLI users to educate themselves about the computational power they are using as well as evaluate the performance of the current LLM. See the effect of prompting techniques.

Our Brainstorming Process: This feature is inspired by our experience using ai in our own coding projects and throughout this course. Our concrete architecture report showed us how much logging already exists in the architecture, unbeknownst to the user. This is what helped us decide on the approaches for this feature.

SAAM: Architectural Alternatives

Approach A

Introduces a dedicated pub-sub event bus that broadcasts typed session events to new subscribers built for recording and replay.

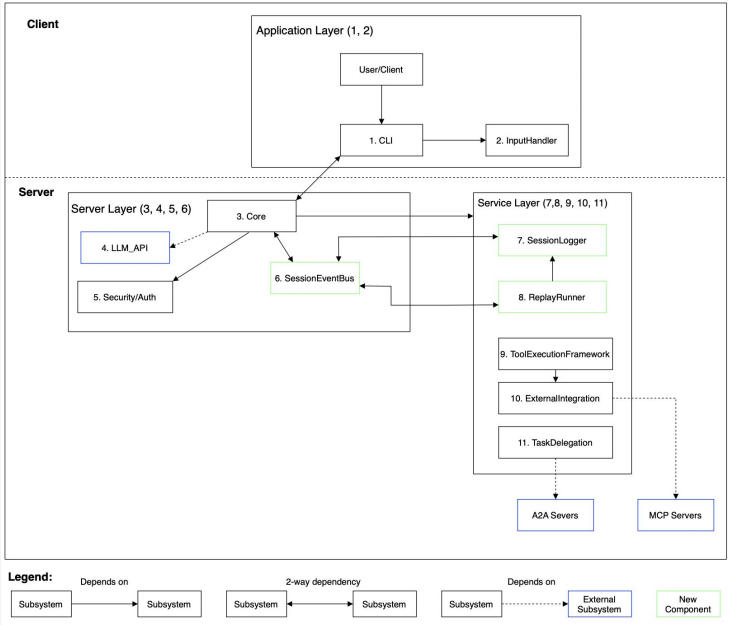


Figure 1. Approach A implementation and impacted subsystems

Approach B

Reuses the existing OpenTelemetry pipeline by adding a new replay-oriented exporter that serialises the spans and logs already being emitted into a file a companion reader can later reconstruct and re-drive.

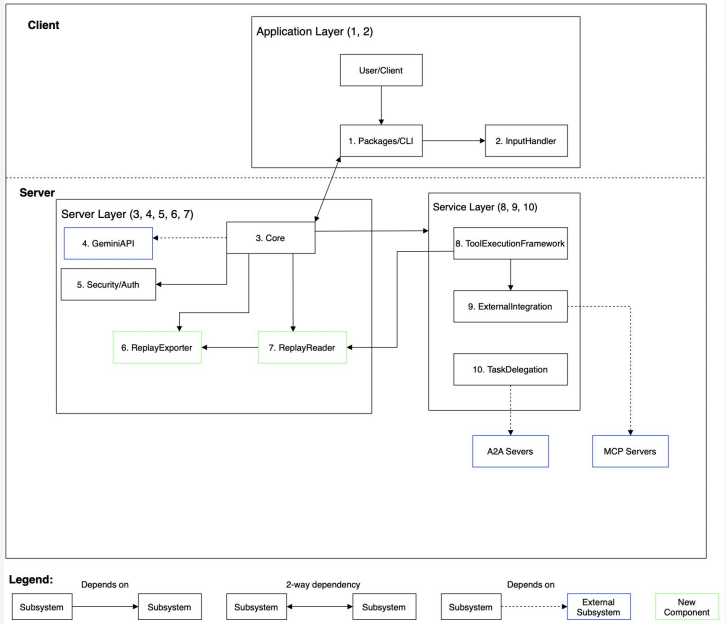


Figure 2. Approach B implementation and impacted subsystems

SAAM: Stakeholders & NFRs

Stakeholders	NFRs
Software Developers: Users of Gemini CLI who use the feature to evaluate model performance and track their technical footprint.	Scalability - feature should work well with multiple files/requirements Performance - feature should not slow down the live CLI session Accuracy - feature must capture every prompt, response, and tool call without omitting or distorting anything
Creaters/ Maintainers of Gemini CLI: Software developers responsible for creating and integrating the replay feature into Gemini CLI.	Security - feature must not introduce vulnerabilities into the codebase Maintainability - implementation must be straightforward to understand, update, and test as the CLI evolves. Testability - feature should be easy to unit/ integration test
Researchers: Users who replay the same sessions for Evaluation or benchmarking	Scalability: feature should handle long, multitool sessions without performance degradation Reliability: feature should work consistently across environments and work for large amounts of data.
Casual Users: Users who want to occasionally revisit or share a session, without deep technical knowledge of the tooling.	Usability - feature should be intuitive for casual users Portability - Recorded sessions should be easily sharable across different machines/environments.

SAAM: Stakeholders & NFRs

NFRs	Approach A	Approach B
Scalability: Researchers, Software Developers	*Moderate-High.* NDJSON scales well, even long sessions can be streamlined. However, large files will need to be stored, which can take noticeable time.	*Moderate.* Telemetry pipeline can handle large amounts of data efficiently for monitoring, but logs aren't structured for exact session replay, making reconstruction less practical.
Accuracy: Software Developers	*High.* This approach is explicitly designed to capture every prompt, tool call, response precisely and in order.	*Moderate - Low.* OpenTelemetry is not meant for exact session replay. It focuses more on tracking system performance, meaning that some details might be missing for performance replay.
Maintainability: Creaters/ maintainers	*High.* This approach ensures a modular architecture with separate components, making the system easy to extend/ modify.	*Moderate.* penTelemetry reduces new code, but tightly couples replay logic to the specific internal data structure and implementation details of telemetry pipeline.
Security: Creaters/ Maintainers	*Moderate-Low*. Captures full session data, which may include sensitive information. Requires additional safeguards.	*High.* Builds on existing telemetry infrastructure, which already enforces secure data handling and limits unnecessary exposure.

SAAM: Advantages & Disadvantages

Approach A

Advantages:

- Pub sub approach allows for more customization when implementing. For example, the event schema can be customized directly.
- ReplayRunner can access core directly, using the same implementation logic and tool logic as live sessions, which is better for comparison

Disadvantages:

- Requires extensive design up front. If the developer misses an event type early on, old log files won't be replayable after schema change
- Implementation of an entire new event bus takes significant man-power

Approach B

Advantages:

- Event capture mechanism already exists in the architecture and tracks session logs, tool execution logic, and tool calls which takes the approach straightforward for debugging
- Aligns with existing patterns/ configurations

Disadvantages:

- logPromps privacy setting can be disabled, meaning that prompt content could be excluded from logs.
- Telemetry layer was designed for observability, not deterministic replay.

Ultimately, we determined the best approach to be **approach A** as it favors high accuracy, scalability, and reliability. We recognize that schema design could falter security, but that is in the hands of the designers.

Effects of Enhancement

- Core ↔ SessionEventBus
- SessionEventBus ↔ SessionLogger
- SessionEventBus ↔ ReplayRunner
- ReplayRunner → SessionLogger

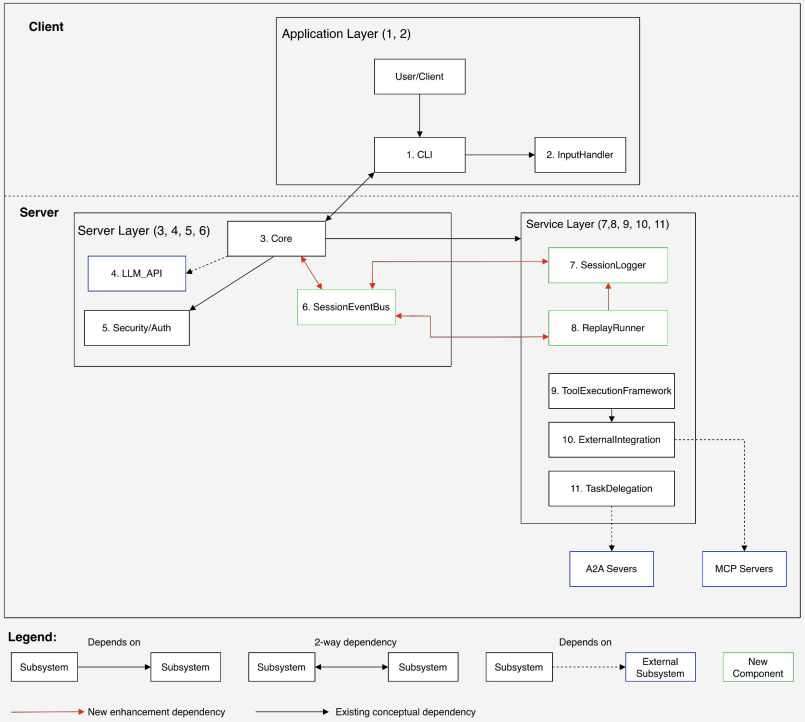


Figure 1: Higher-Level Architecture

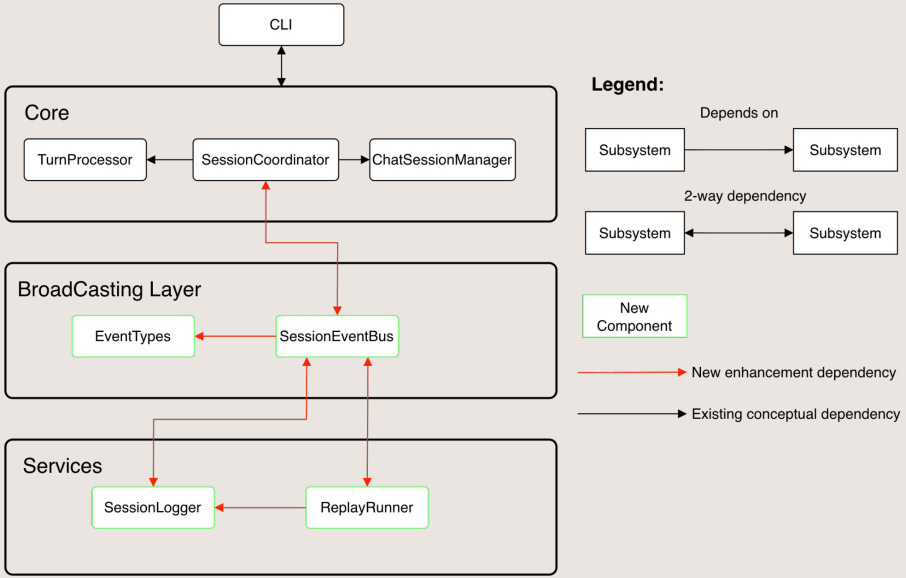
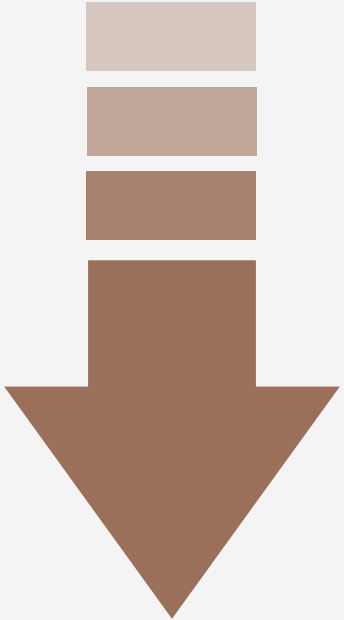


Figure 2: Lower-Level Architecture

- SessionCoordinator ↔ SessionEventBus
- SessionEventBus → EventTypes
- SessionLogger ↔ SessionEventBus
- ReplayRunner ↔ SessionEventBus
- ReplayRunner ↔SessionLogger
- SessionCoordinator → TurnProcessor
- SessionCoordinator → ChatSessionManager

Effects of Enhancement



Maintainability

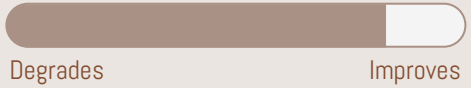
The overall maintainability may decrease due to increased maintenance and logic from new components

Evolvability

The evolvability of Void would improve due to the intro of SessionEventBus, providing strong foundation for future features to use.

Testability

The Pub-Sub style of the feature with SessionEventBus allows other features to be tested in isolation

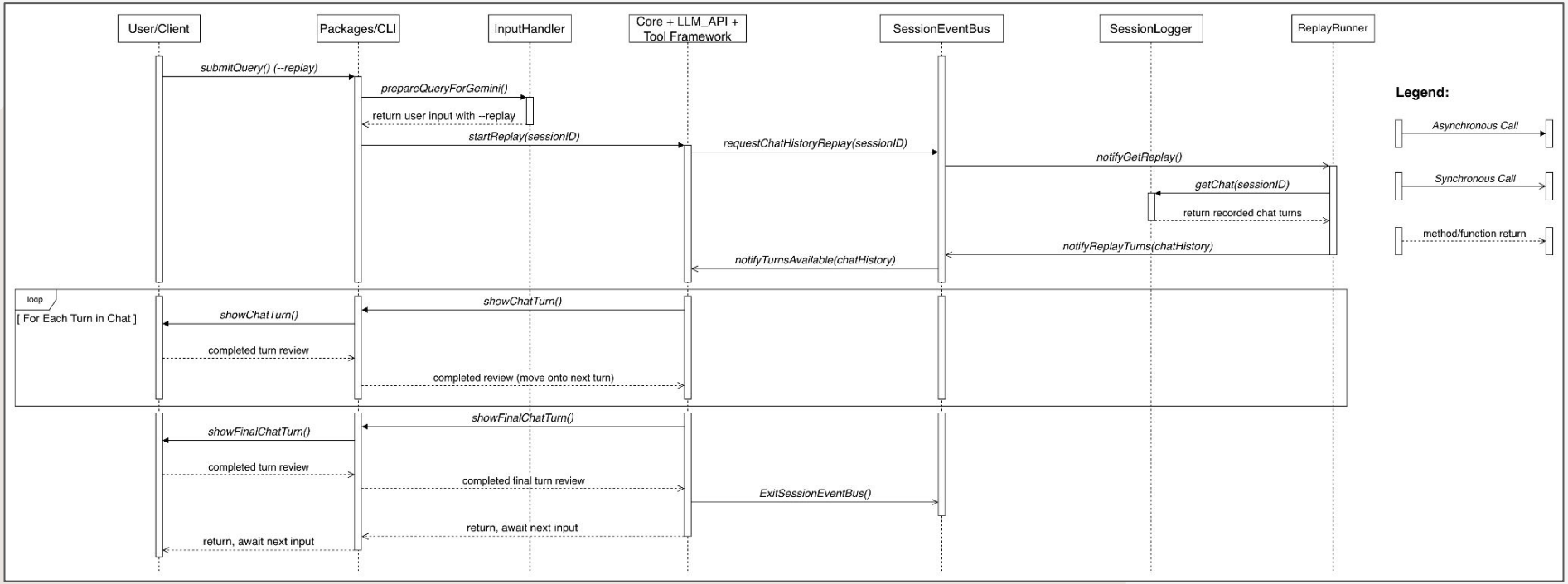


Performance

Gemini CLI's performance may decrease slightly while the replay feature is active, with no decrease in performance when the feature is inactive

Sequence Diagram

A user enables the --replay feature on a recorded session to review the session turn by turn in order to identify a bug introduced by the LLM at some point earlier.



Potential Risks from Enhancement

Incomplete or Inaccurate Session Recording:

- Important events may be missed or broadcast in the wrong order.
- Replay output no longer matches the original session, leading to a loss of trust in the feature.

Performance and Latency Issues:

- Continuous event broadcasting and handling may introduce latency despite the design attempting to prevent performance loss.
- Long or tool-heavy sessions may be especially at risk of loss of performance.

Storage and Scalability Concerns:

- The storage of specific session details, such as tool calls, in NDJSON log files can result in high disk usage.
- Users recording multiple sessions at the same time can exacerbate this issue.

Maintainability and Architectural Complexity:

- The new feature adds components which can add significant complexity if not managed carefully.
- As a result, internal debugging can become harder across core, event, and logging/replay logic

Plans for Testing

Core → SessionEventBus

Test that replay and record events are emitted correctly during a standard CLI session:

- Run a controlled test session with predefined prompts and tool calls.
- Verify that all expected events are broadcast in the correct order.

SessionEventBus → SessionLogger

Test that logged session data is complete and correctly formatted, and written on disk:

- Use mock or controlled sessions and inspect generated NDJSON log file.
- Verify the correctness and accuracy of event fields, such as timestamp and event type.
- Confirm logging has no significant impact on responsiveness of the CLI

ReplayRunner → Recorded Session Logs

Test that ReplayRunner extracts prior session data correctly:

- Replay recorded sessions and compare outputs against the original chat session.
- Replay the same session with a different model and verify both outputs are shown for comparison

Additional

- Test that ReplayRunner extracts prior session data correctly
- Verify the feature is active only when --record or --replay commands are passed
- Verify SessionLogger only active when enabled with commands, ReplayRunner overrides normal session only with --replay

LIMITATIONS & LESSONS LEARNED

Time Limitation

Given additional time, we could have performed additional verification on architectural changes

Required Understanding The System

Deep understanding of the architecture is required even for small relatively additions like ours

Difficulty in identifying Divergences

Mapping all dependencies and potential divergences requires intense review

Our AI Teammate: **perplexity**

Tasks Assigned

- Flesh out ideas on architectural style and subsystems
- Summarize and find sources

Interaction Protocol & Workflow

- Collaborate within partners to craft queries
- Use sero shot and 1-shot prompts
- Refine AI output by asking for specifics

AI agent
January 2026 version

perplexity

Quantitative impact: 12% of final deliverable

Validation & Quality Control

- Ask perplexity for sources
- Fact-check & cross-reference source material

Human-AI Team Dynamic

- Easy to collaborate and get efficient work done
- Importance of verifying AI-generated ideas

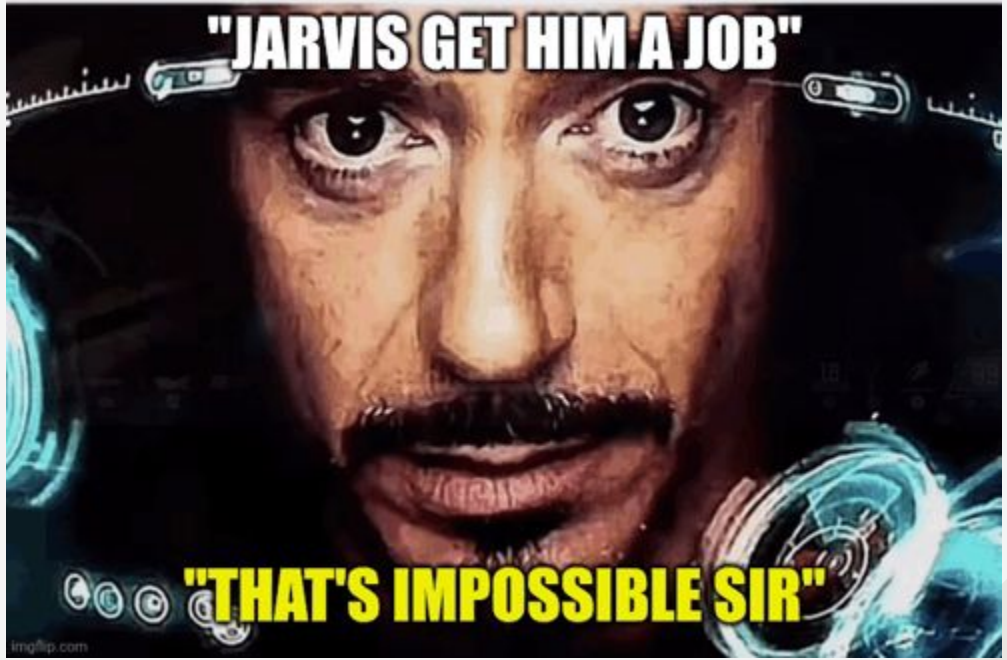
Conclusion

REFERENCES

Anthropic. (2023, March). Claude [May 22 2025 version]. Claude AI. https://claude.ai/

VoidEditor. (n.d.). Void [Computer software]. GitHub. https://github.com/voideditor/void

Thanks for watching!



- The Unemployed*